If we further restrict the acceptable transitions based on proximity along the problem space, on one hand we get a greater number of attempted updates in the area surrounding current point - which could be good, if we are close to the global minimum, as it would speed up convergence.

On the other hand, however, we would likely lose the ability to get out of local minima, which is the best reason to use simulated annealing in the first place.

Therefore, one “safe” solution is to implement method `randomStep` as pure random sampling; this, however, means that we will need a lot of iterations to find a transition downhill, and the algorithm will keep bouncing between valleys, without focusing on fine-tuning. An interesting compromise could be increasing the probability of small steps, but still allowing far-reaching updates, and even trying them every few iterations; this can be done in combination with shrinking the range of the local-search (the small steps) as temperature cools down.

